

An Exact String Matching Algorithm Based upon Selective Matching Order and Branch and Bound Approach

Chia Wei Lu and R. C. T. Lee

Department of Computer Science

National Tsing Hua University, Hsinchu City, Taiwan, ROC

d9762807@oz.nthu.edu.tw, rctlee@rctlee.cyberhood.net.tw

Abstract

In this paper, we propose an improved algorithm for exact string matching problem, which aims to find all the position i 's in a given text where a given pattern occurs. Our algorithm finds the optimal selective matching order of the pattern so that we could have a better performance in the searching phase. To find the optimal matching order, we adapt the branch and bound approach. The experimental results show that our algorithm indeed has the smallest character comparisons for all cases.

1 Introduction

String matching problem is a classical problem in computer science and can be applied to many fields. For example, searching engines, DNA sequence analysis, image processing, subcircuit extraction problem and virus scanning all apply this technique. Let us consider the DNA sequence analysis field. The DNA contains the hereditary information of living things, and it can be regarded as strings composed of four characters, $\{a, c, g, t\}$. We often try to find some specific substring in the DNA strings. Since the DNA strings are always with a huge size, to design efficient string matching algorithms becomes an important research.

In this paper, we are concerned with the exact string matching problem in which we are given a pattern $P = p_1 p_2 \dots p_m$ and a text $T = t_1 t_2 \dots t_n$, $n \geq m$, and we are asked to find all occurrences of P in T . Much research has been done in the area [10][7]. The most remarkable algorithms, KMP [11] and BM [3], have linear searching time in the worst case.

In almost all algorithms, we open a window in T with size m and try to see whether P matches with the substring in this window. A straight forward approach is to compare the substring in the window with P character by character from left to right. If a mismatch is found, move the pattern one step to the right. This approach is an exhaustive search approach because every substring of T is compared, and the time complexity in worst case is $O(mn)$. Nearly all

exact string matching algorithms try to avoid such exhaustive search.

2 The Selection of a Scanning Substring

Let us consider the KMP Algorithm [11] and the BM Algorithm [3]. The KMP Algorithm scans from left to right and terminates as soon as a mismatching is found. Suppose there is a longest suffix U of $W(1, i) = w_1 w_2 \dots w_i$ which is equal to a prefix of P and $z \neq y$ as shown in Figure 1(a), the pattern will be slid to the right as shown in Figure 1(b).

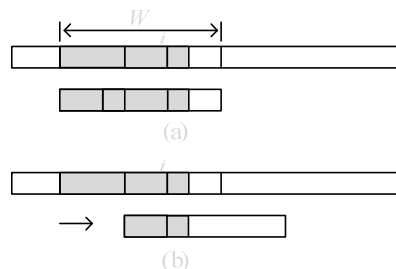


Figure 1. One iteration of the KMP Algorithm.

The BM Algorithm scans from right to left and it also terminates as soon as a mismatch is found. Again, there is a substring U which determines the number of steps to slide the pattern to the right as shown in Figure 2.

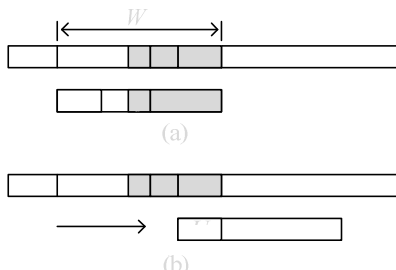


Figure 2. One iteration of the BM Algorithm.

From the above discussion, we can see that the produced substring U is quite important for the

effectiveness of an exact string matching algorithm. Ideally, the substring U should be short. For both KMP and BM algorithms, the substring U is not predetermined because the scanning order is either from the left or from the right. In the following, we shall introduce some algorithms which predetermine the order of scanning.

The Lu's Algorithm [13] Based upon the Uniqueness Property

The Lu's Algorithm [13] is based upon the uniqueness property of a substring. In a string A , a substring $U = A(a,b)$ is unique if there is no substring which is equal to U in $A(1,b-1)$. For instance, consider the string $A = accgtacgta$. The substring $A(4,5) = gt$ is unique while $A(8,9) = gt$ is not unique.

Consider Figure 3(a). In this figure, there is a substring $U = P(a,b)$ in P which matches exactly with a substring $U = W(a,b)$ in W . We also assume that U is unique in P and it is a unique substring selected by the Lu's Algorithm. The uniqueness property tells us that the longest suffix of $W(1,b)$ which is equal to a prefix of P must be contained in U and we can find it by looking at the pattern P only. We may now shift P in such a way that the prefix V in $P(1,b)$ is aligned with the suffix V in $W(1,b)$ as shown in Figure 3(b). The Lu's Algorithm selects an appropriate unique substring.

3 The Sunday's Algorithm

In the Lu's algorithm, we predetermine a substring in P which we use to start the comparison of characters. In the Sunday's Algorithm [15], a sequence is predetermined. This can be explained by considering an example.

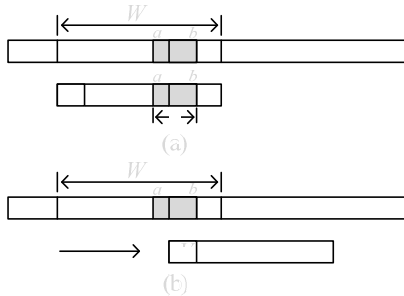


Figure 3. One iteration of the Lu's Algorithm.

Let us consider the data in the following table.

	1	2	3	4	5	6	7	8	9
W	a	t	a	c	t	g	t	t	a
P	a	g	t	t	c	g	c	t	a

Suppose we have determined to use a subsequence, namely $p_6 p_8 p_5$ to start with. Thus our comparison is as follows:

1. Compare $p_6 = g$ with $w_6 = g$. This is an exact match. So we proceed to the next matching.
2. Compare $p_8 = t$ with $w_8 = t$. This is again an exact match. We proceed to the next matching.
3. Compare $p_5 = c$ with $w_5 = t$. They don't match. We have to shift the pattern.

To shift the pattern, we note that we have an exact matching of sequence $p_6 p_8$ in P with sequence $w_6 w_8$ in W . Thus, we examine whether there is a sequence to the left of $p_6 p_8$ which is equivalent to it. We have because $p_2 p_4$ is equivalent to $p_6 p_8$. We have to do a further checking. Since $p_5 = c$ and $w_5 = t$ mismatch, we have to check whether $p_1 = p_5$. We find that $p_1 = a \neq p_5 = c$, we should shift the pattern as shown below.

	1	2	3	4	5	6	7	8	9				
W	a	t	a	c	t	g	t	t	a				
P					a	g	t	t	c	g	c	t	a

Note that this shifting makes sure that p_2 and p_4 exactly match with their corresponding characters in the new window. In the following, we examine a case where the selected sequence is unique in P . Consider the data in the following table.

	1	2	3	4	5	6	7	8	9	10	11							
W	a	g	t	c	a	c	t	t	g	g	a							
P	c	g	a	a	t	g	t	c	g	t	a							
								c	g	a	a	t	g	t	c	g	t	a

We assume that the sequence of exact matches are at locations 7, 9 and 11 and a mismatch occurs at location 6. The matched sequence $p_7 p_9 p_{11}$ however is unique in P because it does not appear to its left.

Although we cannot find a sequence equivalent to $p_7 p_9 p_{11} = tga$ to its left in P , we may try to find whether there is a sequence equivalent to $p_9 p_{11} = ga$ to its left. Of course, this sequence has to be within a prefix. Since the leftmost location of $p_7 p_9 p_{11}$ is 7, we know that the length of this prefix must not be greater than $m - 7 = 11 - 7 = 4$. That is, we search to see whether there is a sequence equivalent to $p_9 p_{11}$ inside $P(1,4)$. There is such a sequence, namely $p_2 p_4 = ga$. Thus we shift the pattern as shown in the above table. Note that the

failure point is out of the range. So, we do not have to consider it.

In the following, we shall introduce how to select the sequence. Let us consider the following pattern.

	1	2	3	4	5	6	7	8	9	10	11
P	a	c	g	c	t	t	g	g	c	t	a

For the first choice, obviously, we should select $p_{11} = a$ to start with because the right most character 'a' is located at location 1 and it will induce a shift of $11-1=10$ steps. The next character chosen should be $p_9 = c$ as the right most character 'c' to it is located in location 4 and this will induce a shift of $9-4=5$ steps. We define a new term $D(i)$.

Given a string $P(1, m)$, $D(i)$, for $1 \leq i \leq m$, is the distance between the right most character equal to p_i to the left of location i if such a character exists in $P(1, i-1)$ and is equal to i if otherwise.

Consider the string in the following table. Its $D(i)$'s are shown also in the table.

i	1	2	3	4	5	6	7	8	9	10	11	12
P	a	c	t	a	g	t	g	c	t	a	g	t
$D(i)$	1	2	3	3	5	3	2	6	3	6	4	3

Having obtained $D(i)$'s, we may now determine the order of comparisons. The location with the largest $D(i)$ will be the first one and that with the second largest $D(i)$ will be the second and so on. Let I be an integer array that is a permutation of $\{1, 2, \dots, m\}$, and $I[j]$ stores the location of the j th character in P that is to be compared. That is, for a window $W = w_1 w_2 \dots w_m$, we compare $p_{I[1]}$ against $w_{I[1]}$ first, and then $p_{I[2]}$ against $w_{I[2]}$ and so on. For example, if we scan the pattern from the left to the right, $I[1]=1$, $I[2]=2$, ..., $I[m]=m$. If we scan the pattern from right to left, $I[1]=m$, $I[2]=m-1$, ..., $I[m]=1$. The table below gives the order of the pattern in the above table.

j	1	2	3	4	5	6	7	8	9	10	11	12
$I[j]$	10	8	5	11	12	9	6	4	3	7	2	1

Having the scanning order, we need to compute the number of characters of window sliding when the first mismatch is found. Suppose we find the first mismatch at $p_{I[j]}$. This means that $p_{I[i]}$ matches with $w_{I[i]}$ for $1 \leq i \leq j-1$. Therefore, we should slide the window to the right so that the characters $w_{I[i]}$'s, for $1 \leq i \leq j-1$, should be aligned with their counterpart in P if $w_{I[i]}$'s are inside the window after sliding. Note that we should have a minimum right shift of the window satisfying the conditions so that we will not miss any solution. The shifting

function Δ used in Sunday's Algorithm [15] is defined as follows. For all $1 \leq j \leq m$, $\Delta[j]$ is the minimum value of $mshift$ such that either $(I[j] - mshift) < 1$, or $p_{I[i]} = p_{(I[i] - mshift)}$, for all $1 \leq i \leq j-1$, and either $(I[j] - mshift) < 1$, or $p_{I[j]} \neq p_{(I[j] - mshift)}$. For example, consider $P = agcca$ and the scanning order $I = \{4, 5, 3, 2, 1\}$. For computing $\Delta[1]$, we find that the minimum value of $mshift = 2$ where $p_{I[1]} = p_4 \neq p_{(I[1]-2)} = p_2$. Therefore $\Delta[1] = 2$. For $\Delta[2]$, we find that the minimum value of $mshift = 1$ where $p_{I[1]} = p_4 = p_{(I[1]-1)} = p_3$ and $p_{I[2]} = p_5 \neq p_{(I[2]-1)} = p_4$. Thus, $\Delta[2] = 1$. For $\Delta[3]$, we find that the minimum value of $mshift = 4$ where $(I[1] - mshift) = 4 - 4 = 0 < 1$, $p_{I[2]} = p_5 = p_{(I[2]-4)} = p_1$, and $(I[3] - mshift) = 3 - 4 = -1 < 1$. Thus, $\Delta[3] = 4$. In this example, it can be verified that $\Delta = \{2, 1, 4, 4, 4\}$. If the scanning is from left to right, Δ is equal to the shift function used in KMP Algorithm. If the scanning is from the right to the left, Δ is equal to one of the shifting functions used in BM Algorithm. Sunday proposed two algorithms [15] which used different scanning orders, and both of them are better than KMP and BM Algorithms. Note that the values of Δ can be computed in preprocessing, and the algorithm for finding the shifting function Δ will not be described in this paper. We refer the readers to [15] for the algorithm.

4 Our Improved Algorithm

Consider the example that $P = aaaaata$ and the alphabet $\Sigma = \{a, c, g, t\}$. The Sunday Algorithm will first match $p_6 = t$ with w_6 of the window. If $p_6 = w_6$, we then can slide the window at least 6 characters to the right for the next window because there is no 't' in $P(1, 5)$. If $p_6 \neq w_6$, we can only slide the window one character to the right because $p_5 \neq p_6$. However, in this example, consider that we match $p_5 = a$ with w_5 first. If $p_5 = w_5$, we can only slide the window one character to the right because $p_4 = p_5 = a$. If $p_5 \neq w_5$, we then can slide the window 5 characters to the right because the characters in $P(1, 5)$ are all equal to the same character 'a'. In this example, the alphabet size is 4, and the probability of the matching two characters is $1/4$ and that of mismatching between two characters is $3/4$. Therefore, using the Maximal Shift Algorithm, we have $1/4$ probability to slide the window 6 characters to the right, and $3/4$ probability to slide the window one character to the right. But, if we match p_5 with w_5 first, we have $1/4$ probability to slide the window for one, and $3/4$ probability to slide the window for 5. The average expected character number of window sliding for matching p_6 first is $(1/4) \cdot 6 + (3/4) \cdot 1 = 9/4$, and that for matching p_5

first is $(1/4) \cdot 1 + (3/4) \cdot 5 = 16/4$. We may conclude that matching p_5 first is better than matching p_6 first.

For a scanning order I , we can compute the shifting function Δ . Assume that the matching fails when we match $p_{I[i]}$. This means that the previous $(i-1)$ matchings were all successful. Therefore, the expected number of shifting is $(1/\sigma)^{i-1} (\sigma - 1/\sigma) \Delta[i]$. To measure the goodness of a scanning order and its Δ , we define a function $AVGS$ to compute the expected number of characters sliding as follows.

$$AVGS(\Delta) = \left(\sum_{i=1}^m \left(\frac{1}{\sigma} \right)^{i-1} \left(\frac{\sigma-1}{\sigma} \right) \Delta[i] \right) + \left(\frac{1}{\sigma} \right)^m \Delta[m+1].$$

Consider the example $P = aaaaata$. A scanning order I_1 and its shifting function Δ_1 is shown in the following table.

j	1	2	3	4	5	6	7	8
$I_1[j]$	5	6	7	4	3	2	1	
$\Delta_1[j]$	5	1	7	6	6	6	6	6

In this example, $AVGS(\Delta_1) = (3/4) \cdot 5 + (1/4) \cdot (3/4) \cdot 1 + (1/4)^2 \cdot (3/4) \cdot 7 + (1/4)^3 \cdot (3/4) \cdot 6 + \dots + (1/4)^6 \cdot (3/4) \cdot 6 + (1/4)^7 \cdot 6 \approx 4.34$. Let us consider another scanning order I_2 and its shifting function Δ_2 used in Sunday's Algorithm is as shown in the following table.

j	1	2	3	4	5	6	7	8
$I_2[j]$	6	7	5	4	3	2	1	
$\Delta_2[j]$	1	7	6	6	6	6	6	6

Then we have $AVGS(\Delta_2) = (3/4) \cdot 1 + (1/4) \cdot (3/4) \cdot 7 + (1/4)^2 \cdot (3/4) \cdot 6 + \dots + (1/4)^7 \cdot 6 \approx 2.43$.

By comparing $AVGS(\Delta_1)$ and $AVGS(\Delta_2)$, we may conclude that the scanning order I_1 is better than I_2 because $AVGS(\Delta_1) > AVGS(\Delta_2)$.

If we can find the scanning order I_{OPT} such that its shifting function has the maximal value of $AVGS$, we would have a better performance in searching phase. However, the number of possible scanning order is the factorial of m . It is not worth to perform an exhaustive searching to find the optimal scanning order. In the following, we give a branch and bound algorithm to efficiently find the optimal scanning order.

Consider the example that $P = aatc$. The scanning order of Sunday's Algorithm is $I = \{4, 3, 2, 1\}$ and its shifting function is $\Delta = \{1, 4, 4, 4, 4\}$. If we use this scanning order, we would obtain an $AVGS=1.75$. We set $AVGS_{opt} = AVGS(\Delta) = 1.75$ as a lower bound of optimal $AVGS$, $AVGS_{opt}$. Then we use a branch and bound strategy for finding the optimal scanning order as illustrated in Figure 4.

Consider the first level of the tree in the Figure 4, i.e. $i=1$. Any one of the characters of P , p_1 , p_2 , p_3 , or p_4 , can be the first character being compared in a window. We compute the Δ_j 's for the every possibility. Consider the case that p_2 is the first character being compared. If p_2 mismatches with its corresponding character in a window, we can slide the window 2 characters to the right because $p_1 = p_2 = a$. Therefore, we set $\Delta_2[1] = 2$. As for the values of $\Delta_2[2..5]$, we temporarily set all of them being $m=4$ for computing the expected largest value of $AVGS(\Delta_2)$. That is, if $AVGS(\Delta_2)$ is smaller than $AVGS_{opt} = 1.75$, we then can stop this branch. In this example, only the branch $j=2$ will be branched. In the second level, $i=2$, under the situation that p_2 is the first character being compared, there are three possibility of the second character being compared, p_1 , p_3 , and p_4 . For each possibility, we update the values of Δ_j 's. We repeat the processes and if it can be branched to the level $i=m$, we then compute the shifting function. If $AVGS(\Delta) > AVGS_{opt}$, we then set $AVGS_{opt} = AVGS(\Delta)$ and record this better scanning order. In this example, we reach the level $i=m$, and the value of $AVGS(\Delta)$ is 1.94 which is larger than $AVGS_{opt} = 1.75$. Thus, we set $AVGS_{opt} = 1.94$ and record the scanning order $I = \{2, 4, 3, 1\}$ which is better than that of Sunday's Algorithm. Note that there are still three branches which have not been branched. However, we have set $AVGS_{opt} = 1.94$, and all of the branches will stop immediately because $AVGS(\Delta_j) \leq AVGS_{opt} = 1.94$ now.

Our algorithm is described in **Algorithm 1**.

5 Experiments

In our experiments, we tested the Sunday's Algorithm by using our optimal scanning order and Sunday's original scanning order respectively. We randomly generated the text T of size $n=10M$ and the pattern of size $m \in \{5, 10, 20\}$ by using the alphabets $\Sigma_1 \in \{a, b\}$ and $\Sigma_2 \in \{a, c, g, t\}$ respectively. We tested the performance by comparing the number of characters comparison. Additionally, we also tested and compared the results with a brute force algorithm. The results are shown in Table 1.

The experimental results show that using our optimal scanning indeed has the smallest number of character comparison in all cases.

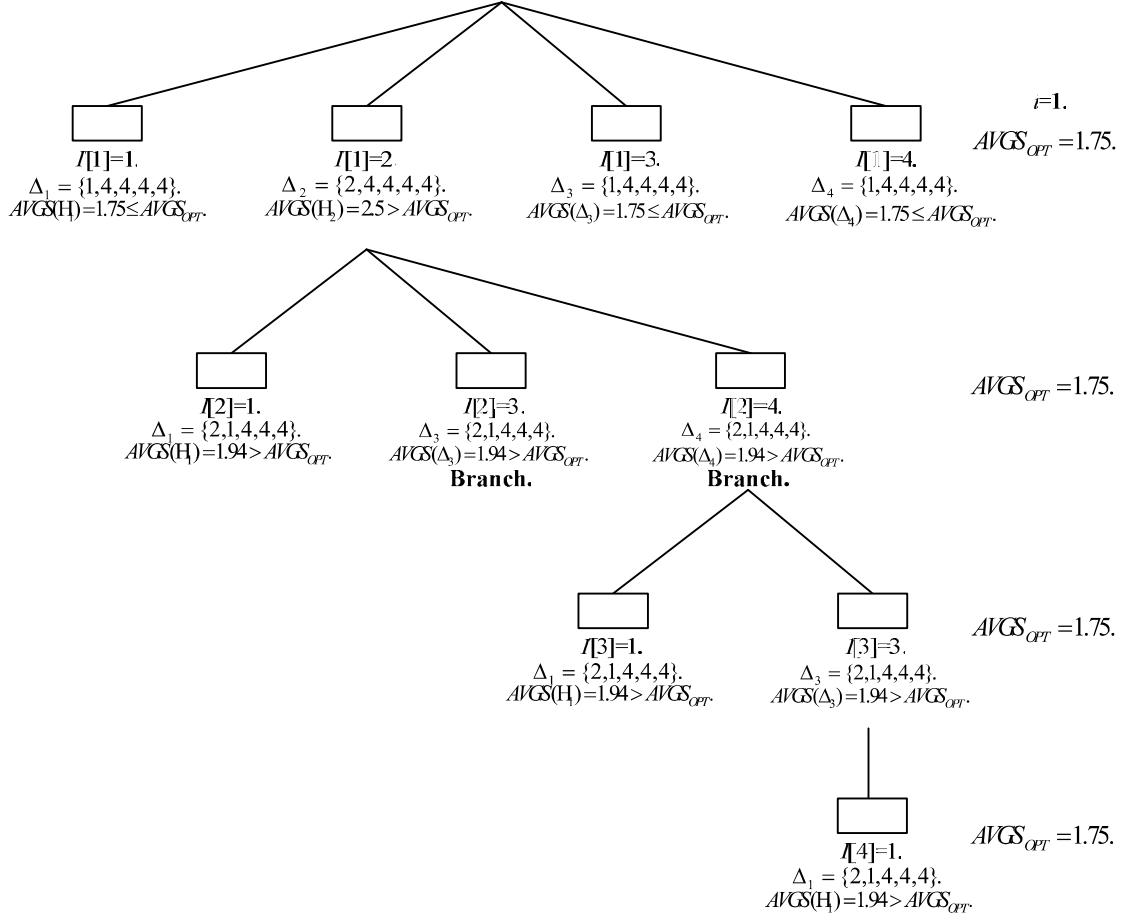


Figure 4. A branch and bound algorithm for finding the optimal scanning order.

Algorithm 1 A branch and bound algorithm to find the optimal scanning order

Input: A pattern P and the alphabet Σ .

Output: The optimal scanning order I_{OPT} and Δ_{OPT} .

- 1: Compute the scanning order I and its shifting function Δ used in Sunday's Algorithm. Set $AVGS_{OPT} = AVGS(\Delta)$.
 - 2: $I_{OPT} = \text{FindOpt_branch_and_bound}(1, I, AVGS_{OPT})$.
 - 3: Return I_{OPT} and its shifting function Δ_{OPT} .
-

FindOpt_branch_and_bound($i, I, AVGS_{OPT}$)

Input: An integer i , an integer array I , and an integer $AVGS_{OPT}$.

Output: The optimal scanning order I_{OPT} .

```

1:  if  $i \leq m$  then
2:      For every  $j$ , where  $1 \leq j \leq m$ , such that  $j \neq I[k]$  for all  $k < i$ , set  $I_j[i] = j$  and compute the
        shifting function  $\Delta_j[1..i]$ . Set the values of  $\Delta_j[i+1..m+1] = m$ .
3:      while ( $AVGS(\Delta_j) > AVGS_{OPT}$  for some  $j$ ) do                                /*bound */
4:          Find the  $j_{max}$  such that  $AVGS(\Delta_{j_{max}})$  is the largest among all  $1 \leq j_{max} \leq m$ , and set
             $AVGS(\Delta_{j_{max}}) = 0$ .
5:           $I_{OPT} = \text{FindOpt\_branch\_and\_bound}(i+1, I_{j_{max}}, AVGS_{OPT})$ .          /*branch */
6:      end of while
7:  end of if
8:  else
9:      Compute the shifting function  $\Delta$  of  $I$ .
10:     if  $AVGS(\Delta) > AVGS_{OPT}$  then
11:         Set  $AVGS_{OPT} = AVGS(\Delta)$  and  $I_{OPT} = I$ .
12:     end of if
13: end of else
14: Return  $I_{OPT}$ .

```

Table 1. The number of character comparisons in the searching phase

Alphabet Size	m	S90	Our Algorithm	BF
2	5	8465946	7857099	19375856
	10	5976810	4910375	19980690
	20	4373341	2941580	19999977
4	5	7413540	6320276	13320778
	10	5302279	4242591	13333300
	20	3752596	2813221	13333316

6 Conclusion and Future Research

In this paper, we propose a branch and bound algorithm to find the optimal matching order so that our exact string matching algorithm has a smaller number of character comparisons in the searching phase. The experimental results show that our optimal matching order is better than that of Sunday's Algorithm. It will be interesting future work to analyze the average-case time complexity of our algorithm.

Reference

- [1] Apostolico, A. and Crochemore, M., Optimal canonization of all substrings of a string, Information and Computation, Vol. 95, 1991, pp. 760-795.
- [2] Apostolico, A. and Giancarlo, R., The Boyer-Moore-Galil string searching strategies revisited, SIAM Journal on Computing, Vol. 15, 1986, pp. 980-105.
- [3] Boyer, R.S. and Moore, J.S., A fast string searching algorithm, Communications of the ACM, Vol. 20, 1977, pp. 762-772.
- [4] Colussi, L., Correctness and efficiency of the pattern matching algorithms, Information and Computation, Vol. 95, No. 2, 1991, pp. 225-251.
- [5] Colussi, L., Fastest pattern matching in strings, Journal of Algorithms, Vol. 16, No. 2, 1994, pp.

- 1630189.
- [6] Crochemore, M., Czumaj, A., Gasieniec, L., Jarominek, S., Lecroq, T., Plandowski, W. and Rytter, W., Speeding up two string matching algorithms, *Algorithmica*, Vol. 12, 1994, pp. 2470267.
 - [7] Charras, C., Lecroq, T. and Pehoushek, J.D., A very fast string matching algorithm for small alphabets and long patterns, in *Proceedings of the 9th Annual Symposium on Combinatorial Pattern Matching*, M. Farach-Colton ed., Piscataway, New Jersey, Lecture Notes in Computer Science 1448, 1998, pp. 55064, Springer-Verlag, Berlin.
 - [8] Galil, Z. and Giancarlo, R., On the exact complexity of string matching: upper bounds, *SIAM Journal on Computing*, Vol. 21, No. 3, 1992, pp. 4070437.
 - [9] Galil, Z. and Seiferas, J., Time-space optimal string matching, *Journal of Computer and System Science*, Vol. 26, No.3, 1983, pp. 2800294.
 - [10] Horspool, R.N., Practical fast searching in strings, *Software - Practice & Experience*, Vol. 10, No. 6, 1980, pp. 5010506.
 - [11] Knuth, D.E., Morris (Jr), J.H. and Pratt, V.R., Fast pattern matching in strings, *SIAM Journal on Computing*, Vol. 6, No. 2, 1977, pp. 3230350.
 - [12] Lecroq, T., A variation on the Boyer-Moore algorithm, *Theoretical Computer Science*, Vol. 92, No. 1, 1992, pp. 1190144.
 - [13] Chia Wei Lu and R. C. T. Lee, String Matching Algorithms Based upon the Uniqueness Property, *Proc. of the 24th Workshop on Combinatorial Mathematics and Computation Theory*, April 27028, 2007, pp. 3850392.
 - [14] Morris (Jr), J.H. and Pratt, V.R., A linear pattern-matching algorithm, Technical Report 40, 1970, University of California, Berkeley.
 - [15] Sunday, D.M., A very fast substring search algorithm, *Communications of the ACM*, Vol. 33, No. 8, 1990, pp. 1320142.
 - [16] Smith, P.D., Experiments with a very fast substring search algorithm, *Software - Practice & Experience*, Vol. 21, No. 10, 1991, pp. 106501074.
 - [17] Simon, I., String matching algorithms and automata, in *Proceedings of 1st American Workshop on String Processing*, R.A. Baeza-Yates and N. Ziviani ed., 1993, pp. 1510157, Universidade Federal de Minas Gerais, Brazil.